

Bloque 1 — Mini-especificaciones funcionales y técnicas

Fundamentos transversales: A001 a A007

1. Propósito del bloque

Este bloque define las bases funcionales, técnicas y visuales que deberán respetar los módulos de:

- Rendiciones y fondos por rendir.
- Banco y conciliación bancaria.
- Facturación y cobranza.
- Factoring.
- Alertas, reportes y dashboards.

No se construyen todavía los procesos completos de cada módulo. El objetivo es dejar preparado el esqueleto común sobre el cual se implementarán las tareas restantes de la Carta Gantt.

El archivo `models2-administracion.py` ya delimita correctamente la fase inicial a banco, conciliación, facturación, cobranza, factoring y rendiciones. La contabilidad general, los centros de costo avanzados y los asientos contables automáticos quedan fuera de esta etapa.

2. Resumen del bloque

Código	Actividad	Horas Gantt	Cobertura actual	Resultado principal
A001	Levantamiento y alcance funcional	20 h	Documental	Especificación funcional y alcance
A003	Estados y reglas de negocio	6 h	Parcialmente cubierta	Matriz de estados y transiciones
A002	Modelo de datos y migraciones	12 h	Parcialmente cubierta	Modelos revisados y migraciones estables
A004	Permisos y roles	10 h	Parcialmente cubierta	Matriz de permisos y protección de vistas
A005	Auditoría e historial	10 h	Parcialmente cubierta	Trazabilidad técnica y funcional
A006	Gestión de archivos adjuntos	8 h	Parcialmente cubierta	Carga y descarga segura de documentos
A007	Diseño UX y navegación	12 h	Sin dependencia de nuevos modelos	Plantilla base y navegación del módulo

El bloque suma **78 horas**.

La secuencia recomendada de ejecución es:

```
A001 → A003 → A002 → A004 → A005 → A006 → A007
```

Aunque los códigos numéricos sitúan A002 antes de A003, resulta más seguro definir primero los estados, transiciones y reglas relevantes para que el modelo de datos definitivo los represente correctamente.

A001 tiene 20 horas y exige documentar procesos, actores, excepciones y criterios de aceptación. A002 tiene 12 horas para validar modelos, relaciones, restricciones e índices.

3. Decisiones arquitectónicas transversales

3.1 Aplicación Django principal

Se utilizará una aplicación denominada, provisionalmente:

```
administracion
```

La aplicación contendrá los procesos financieros y administrativos, mientras que la aplicación existente:

```
extintores
```

seguirá administrando clientes, intervenciones, ODT, técnicos, productos y servicios.

La relación principal entre ambas aplicaciones será:

```
administracion.FacturaVenta
      ↓
extintores.Cliente
```

No se duplicará el modelo de cliente en la aplicación de Administración.

3.2 Separación de responsabilidades

Las vistas no deberán contener directamente toda la lógica del negocio.

Se aplicará esta distribución:

```
Vista
  ↓
Formulario
  ↓
Servicio de dominio
  ↓
Modelo y base de datos
```

Para consultas complejas:

```
Vista
  ↓
Selector
  ↓
QuerySet
```

Responsabilidad de cada capa

Capa	Responsabilidad
<code>models.py</code>	Estructura, relaciones, propiedades y validaciones básicas
<code>forms.py</code>	Validación de entrada de usuario
<code>services.py</code>	Operaciones de negocio y cambios de estado
<code>selectors.py</code>	Consultas, filtros y agregaciones
<code>views.py</code>	Coordinación de la solicitud HTTP
<code>permissions.py</code>	Reglas de acceso
<code>templates/</code>	Presentación
<code>tests/</code>	Validación automática

Las operaciones que cambien saldos, estados financieros o relaciones entre pagos deberán utilizar:

```
transaction.atomic()
```

y, cuando dos operaciones puedan modificar simultáneamente el mismo registro:

```
select_for_update()
```

3.3 Estructura inicial de carpetas

```
administracion/
├── __init__.py
├── apps.py
├── urls.py
├── admin.py
├──
├── models/
│   ├── __init__.py
│   ├── auditoria.py
│   ├── bancos.py
│   ├── facturacion.py
│   ├── factoring.py
│   ├── rendiciones.py
│   └── adjuntos.py
├──
├── forms/
│   ├── __init__.py
│   └── adjuntos.py
├──
├── views/
│   ├── __init__.py
│   ├── inicio.py
│   ├── auditoria.py
│   └── adjuntos.py
├──
├── services/
│   ├── __init__.py
│   ├── auditoria.py
│   ├── estados.py
│   └── adjuntos.py
├──
├── selectors/
│   ├── __init__.py
│   └── auditoria.py
├──
├── permissions.py
├── context_processors.py
├──
├── templates/administracion/
│   ├── base_administracion.html
│   ├── inicio.html
│   ├── componentes/
│   │   ├── breadcrumbs.html
│   │   ├── mensajes.html
│   │   ├── paginacion.html
│   │   ├── estado_badge.html
│   │   └── tabla_vacia.html
```

```

|   |   └─ adjuntos.html
|   └─ auditoria/
|       └─ historial.html
|       └─ detalle_evento.html
|
|   └─ static/administracion/
|       └─ css/
|           └─ administracion.css
|       └─ js/
|           └─ administracion.js
|           └─ adjuntos.js
|
|   └─ migrations/
|   └─ tests/
|       └─ test_estados.py
|       └─ test_permisos.py
|       └─ test_auditoria.py
|       └─ test_adjuntos.py

```

La división física de `models.py` es recomendada, pero no obliga a cambiar la aplicación ni los nombres de las tablas actuales.

A001 — Levantamiento y alcance funcional

1. Objetivo

Documentar cómo funciona actualmente la administración del negocio y cómo deberá operar el nuevo sistema.

Esta tarea no debe producir código funcional. Su resultado será la fuente de verdad para las tareas posteriores.

2. Alcance funcional

El levantamiento debe cubrir:

- Actores.
- Procesos actuales.
- Procesos esperados.
- Documentos de entrada.
- Resultados o documentos de salida.
- Estados.
- Excepciones.
- Aprobaciones.
- Relaciones entre módulos.
- Reglas de seguridad.
- Criterios de aceptación.

3. Actores del sistema

Los actores funcionales no deben confundirse con los recursos de la Carta Gantt.

Los recursos:

Analista, Backend, Frontend, Integraciones, QA y Documentación

representan tipos de trabajo de desarrollo.

Los roles funcionales propuestos para el sistema son:

Rol funcional	Responsabilidad
Administrador del sistema	Configuración general y usuarios
Encargado de Finanzas	Bancos, facturas, pagos y conciliación
Revisor de rendiciones	Revisar y observar gastos
Aprobador	Aprobar o rechazar rendiciones y cierres
Responsable de rendición	Presentar gastos y comprobantes
Encargado de cobranza	Seguimiento de facturas y clientes
Auditor o consulta	Acceso de solo lectura e historial

Inicialmente una misma persona puede tener varios roles.

4. Procesos que deben documentarse

Banco

Registrar banco
→ Registrar cuenta
→ Importar cartola
→ Crear movimientos
→ Clasificar
→ Aplicar
→ Conciliar
→ Cerrar período

Facturación

Registrar factura
→ Controlar vencimiento
→ Registrar pago
→ Aplicar a una o varias facturas

- Actualizar cobranza
- Conciliar con cartola

Rendiciones

- Entregar fondo
- Crear rendición
- Registrar gastos
- Adjuntar documentos
- Presentar
- Revisar
- Aprobar
- Liquidar
- Devolver o reembolsar
- Conciliar

Factoring

- Ceder factura
- Registrar anticipo
- Controlar cobro
- Detectar mora o pago directo
- Liquidar o devolver
- Conciliar

5. Reglas generales iniciales

1. Ningún pago se considerará conciliado sin un movimiento bancario asociado, salvo formas de pago expresamente autorizadas.
2. Ninguna rendición podrá liquidarse antes de ser aprobada.
3. Una factura podrá recibir múltiples pagos.
4. Un pago podrá distribuirse entre múltiples facturas.
5. Un movimiento bancario podrá aplicarse a múltiples destinos.
6. Ninguna aplicación podrá superar el monto disponible del origen.
7. Las operaciones anuladas no deberán eliminarse físicamente cuando formen parte de un proceso financiero.
8. Las modificaciones críticas deberán dejar historial.
9. Los comprobantes financieros no deberán quedar accesibles mediante URLs públicas sin validación de permisos.

6. Documentos de salida

```
docs/administracion/  
├─ 01_alcance.md  
├─ 02_actores.md  
├─ 03_procesos.md  
├─ 04_reglas_negocio.md  
├─ 05_excepciones.md  
├─ 06_integraciones.md  
├─ 07_criterios_aceptacion.md  
└─ 08_glosario.md
```

7. Criterios de aceptación

A001 se considerará terminada cuando:

- Todos los módulos tengan un objetivo definido.
- Se hayan identificado los actores.
- Cada proceso tenga inicio y resultado.
- Las excepciones principales estén documentadas.
- Los límites de esta fase estén explícitos.
- No existan términos financieros ambiguos.
- Cada una de las 79 tareas tenga correspondencia con uno o más procesos.
- El documento sea suficiente para diseñar estados y tablas sin inventar reglas durante la programación.

8. Observación sobre las horas

Las 20 horas son razonables para documentar una primera versión funcional, siempre que las decisiones principales del negocio ya hayan sido discutidas.

A003 — Estados y reglas de negocio

1. Objetivo

Construir una matriz única de estados, transiciones permitidas, responsables, validaciones y efectos secundarios.

La Carta Gantt exige una matriz de estados, transiciones y excepciones validada.

2. Cobertura actual

Los modelos ya contienen estados para:

- Importaciones de cartola.
- Movimientos bancarios.

- Conciliaciones.
- Facturas.
- Pagos.
- Aplicaciones.
- Operaciones de factoring.
- Alertas.
- Rendiciones.
- Revisiones y aprobaciones.
- Devoluciones.

La estructura existe, pero actualmente los estados están distribuidos entre numerosos `TextChoices`.
Falta centralizar las reglas de transición.

3. Regla arquitectónica

Las vistas y formularios no deberán cambiar estados directamente mediante:

```
objeto.estado = "APROBADA"
objeto.save()
```

Los cambios deberán realizarse mediante servicios explícitos:

```
presentar_rendicion(...)
aprobar_rendicion(...)
rechazar_rendicion(...)
liquidar_rendicion(...)
cerrar_conciliacion(...)
anular_pago(...)
registrar_pago_factor(...)
```

4. Matrices iniciales

Rendición

Estado actual	Acción	Nuevo estado	Condición
BORRADOR	Presentar	PRESENTADA	Debe tener detalles
PRESENTADA	Iniciar revisión	EN_REVISION	Usuario con permiso
EN_REVISION	Observar	OBSERVADA	Comentario obligatorio
OBSERVADA	Corregir y presentar	PRESENTADA	Responsable
EN_REVISION	Aprobar	APROBADA	Sin observaciones pendientes
EN_REVISION	Rechazar	RECHAZADA	Motivo obligatorio

Estado actual	Acción	Nuevo estado	Condición
APROBADA	Liquidar	LIQUIDADA	Cálculos validados
LIQUIDADA	Cerrar	CERRADA	Diferencia regularizada
CERRADA	Reabrir	Estado definido por servicio	Permiso especial y motivo

Movimiento bancario

Estado actual	Condición	Estado calculado
Sin aplicaciones	Monto aplicado = 0	NO_CONCILIADO
Aplicación incompleta	Aplicado menor que monto	PARCIAL
Aplicación completa	Aplicado igual al monto	CONCILIADO
Exclusión vigente	Motivo autorizado	EXCLUIDO

El modelo `MovimientoBancario` ya calcula su estado a partir del monto aplicado; esta lógica deberá invocarse después de crear, modificar o anular aplicaciones.

Factura

Condición	Estado de cobranza
Documento anulado	ANULADA
Factura cedida activa	CEDIDA_FACTORING
Sin pagos y vigente	PENDIENTE
Sin pagos y vencida	VENCIDA
Pago incompleto	PARCIAL
Total pagado	PAGADA

Pago de cliente

Condición	Estado
Sin aplicaciones	REGISTRADO
Aplicación menor que el pago	PARCIAL
Aplicación igual al pago	APLICADO
Pago revertido	ANULADO

5. Archivos técnicos

```
administracion/  
├── services/  
│   ├── estados.py  
│   ├── rendiciones.py  
│   ├── pagos.py  
│   ├── conciliacion.py  
│   └── factoring.py  
└── tests/  
    └── test_transiciones_estado.py
```

6. Interfaz

Las pantallas deberán mostrar:

- Badge del estado.
- Acciones permitidas.
- Acciones bloqueadas.
- Motivo del bloqueo.
- Historial de transiciones.
- Confirmación antes de transiciones irreversibles.

Ejemplo:

Estado: EN REVISIÓN

[Observar] [Aprobar] [Rechazar]

Los botones se mostrarán según permisos y estado, pero la validación definitiva deberá efectuarse siempre en el backend.

7. Pruebas

Por cada transición:

1. Transición válida.
2. Transición inválida.
3. Usuario con permiso.
4. Usuario sin permiso.
5. Campos obligatorios.
6. Doble ejecución.
7. Error dentro de la transacción.
8. Registro de auditoría.

8. Criterios de aceptación

- Todos los estados cuentan con definición.
- Cada transición identifica origen y destino.
- No existen cambios directos de estado desde templates.
- Las transiciones inválidas generan un error controlado.
- Las operaciones financieras se ejecutan dentro de transacciones.
- Las transiciones críticas generan auditoría.
- Las pruebas cubren caminos permitidos y rechazados.

9. Observación sobre las horas

Las 6 horas alcanzan para definir la matriz y crear el esqueleto de servicios, pero no para implementar y probar completamente todas las transiciones de los cuatro módulos.

La implementación detallada se distribuirá en las tareas específicas de Rendiciones, Banco, Facturación y Factoring.

A002 — Modelo de datos y migraciones

1. Objetivo

Verificar que la base de datos represente correctamente los procesos aprobados en A001 y A003.

La tarea debe asegurar que modelos, relaciones, restricciones e índices migren sin errores.

2. Cobertura actual

`models2-administracion.py` ya incorpora una base de auditoría abstracta con:

- `creado_en`.
- `actualizado_en`.
- `creado_por`.
- `actualizado_por`.

También contiene los modelos principales para:

- Bancos.
- Cuentas.
- Cartolas.
- Movimientos.
- Facturas.
- Pagos.
- Factoring.
- Rendiciones.
- Aplicaciones bancarias.

La cobertura es alta, pero no completa.

3. Ajustes requeridos

3.1 Separar el archivo de modelos

El archivo de Administración supera ampliamente una estructura cómoda de mantenimiento. Se recomienda dividirlo en módulos sin cambiar las tablas:

```
models/  
├─ auditoria.py  
├─ bancos.py  
├─ facturacion.py  
├─ factoring.py  
├─ rendiciones.py  
└─ adjuntos.py
```

`models/__init__.py` deberá importar las clases públicas para mantener los imports actuales.

3.2 Revisar señales del módulo Extintores

Actualmente existen dos receptores `post_save` relacionados con la creación de `TechnicianProfile`. Ambos pueden intentar crear el perfil del técnico. Además, un `post_save` de usuario no es el mecanismo correcto para detectar que el usuario fue agregado posteriormente a un grupo.

Se deberá:

- Eliminar la duplicación.
- Crear el perfil desde un servicio explícito o una señal `m2m_changed`.
- Usar `get_user_model()` de forma consistente.
- Agregar una prueba que impida perfiles duplicados.

3.3 Nuevos modelos previstos

Los siguientes modelos serán definidos en las tareas transversales:

```
EventoAuditoria  
AdjuntoAdministracion
```

Otros modelos, como reglas de conciliación automática, se analizarán en su tarea específica.

3.4 Restricciones

Se comprobarán:

- Montos positivos.
- Rangos de fecha válidos.
- Identificadores únicos.
- Unicidad de cartolas por hash.

- Unicidad de movimientos por fingerprint.
- Un solo destino válido en aplicaciones.
- Integridad entre clientes, pagos y facturas.
- Protección de registros financieros referenciados.

3.5 Índices

Se revisarán índices en:

- Estados.
- Fechas.
- Número de factura.
- Cliente.
- Cuenta bancaria.
- Período.
- Estado de conciliación.
- Fecha de vencimiento.
- Número de rendición.
- Estado de rendición.

4. Migraciones

El procedimiento será:

1. Respaldar base de datos
2. Ejecutar `makemigrations --check`
3. Crear migraciones por cambio lógico
4. Revisar SQL con `sqlmigrate`
5. Probar sobre copia de la base
6. Ejecutar `migrate`
7. Ejecutar pruebas
8. Validar datos existentes

No deberán agruparse cambios no relacionados en una única migración.

5. Comandos de validación

```
python manage.py check
python manage.py makemigrations --check
python manage.py showmigrations
python manage.py migrate --plan
python manage.py test administracion
```

6. Vistas y templates

A002 no necesita una pantalla funcional propia.

Se utilizará inicialmente Django Admin para verificar:

- creación;
- edición;
- relaciones;
- restricciones;
- visualización de datos.

7. Criterios de aceptación

- `python manage.py check` no presenta errores.
 - Las migraciones se aplican sobre una base limpia.
 - Las migraciones se aplican sobre una copia de la base actual.
 - Los nombres de tablas existentes se conservan.
 - No se pierden datos.
 - Las restricciones rechazadas generan errores comprensibles.
 - Los modelos pueden importarse sin dependencias circulares.
 - Django Admin permite inspeccionar los modelos principales.
 - Las pruebas de integridad pasan.
-

A004 — Permisos y roles

1. Objetivo

Garantizar que cada usuario pueda ver y ejecutar únicamente las acciones correspondientes a su función.

La Carta Gantt exige verificar accesos permitidos y denegados por rol y acción.

2. Cobertura actual

El proyecto ya utiliza:

- `User`.
- `Group`.
- Perfiles relacionados con usuarios.
- Relaciones `creado_por` y `actualizado_por`.

Sin embargo, todavía falta una matriz integral de permisos para Administración y Finanzas.

3. Roles propuestos

Administrador del sistema

Puede:

- Administrar usuarios y grupos.

- Configurar bancos y cuentas.
- Reabrir procesos.
- Consultar auditoría.
- Ejecutar acciones extraordinarias.

Finanzas

Puede:

- Importar cartolas.
- Registrar movimientos.
- Registrar facturas y pagos.
- Aplicar movimientos.
- Conciliar.
- Gestionar factoring.
- Generar reportes.

Revisor de rendiciones

Puede:

- Consultar rendiciones presentadas.
- Revisar detalles.
- Observar gastos.
- Aprobar o rechazar, según la política final.

Responsable de rendición

Puede:

- Crear sus rendiciones.
- Editar borradores.
- Adjuntar comprobantes.
- Presentar.
- Corregir rendiciones observadas.
- Consultar su historial.

No puede:

- Aprobar su propia rendición.
- Modificar rendiciones de otros responsables.
- Editar una rendición cerrada.

Auditor o consulta

Puede:

- Ver registros.
- Ver historiales.
- Descargar reportes autorizados.

No puede modificar datos.

4. Permisos Django

Se utilizarán permisos estándar:

```
add
view
change
delete
```

y permisos personalizados:

```
permissions = [
    ("presentar_rendicion", "Puede presentar rendiciones"),
    ("revisar_rendicion", "Puede revisar rendiciones"),
    ("aprobar_rendicion", "Puede aprobar rendiciones"),
    ("reabrir_rendicion", "Puede reabrir rendiciones"),
    ("cerrar_conciliacion", "Puede cerrar conciliaciones"),
    ("reabrir_conciliacion", "Puede reabrir conciliaciones"),
    ("anular_pago", "Puede anular pagos"),
    ("ver_auditoria", "Puede consultar auditoría"),
]
```

5. Protección de vistas

Se utilizarán:

```
LoginRequiredMixin
PermissionRequiredMixin
```

o decoradores equivalentes.

Ejemplo conceptual:

```
URL
→ autenticación
→ permiso de módulo
→ permiso de acción
→ validación sobre el objeto
→ servicio
```

6. Permisos por objeto

Django no proporciona por defecto permisos complejos por instancia. Se implementarán reglas explícitas:

```
puede_editar_rendicion(usuario, rendicion)
puede_aprobar_rendicion(usuario, rendicion)
puede_descargar_adjunto(usuario, adjunto)
puede_reabrir_conciliacion(usuario, conciliacion)
```

7. Templates

Los templates ocultarán botones no autorizados:

```
{% if perms.administracion.aprobar_rendicion %}
    <button>Aprobar</button>
{% endif %}
```

Esto solo mejora la interfaz. El backend deberá volver a comprobar el permiso.

8. Archivos

```
administracion/
├── permissions.py
├── management/
│   ├── commands/
│   │   └── configurar_rol_administracion.py
└── tests/
    └── test_permisos.py
```

El comando de configuración deberá crear o actualizar los grupos sin generar duplicados.

9. Pruebas

La matriz mínima será:

Caso	Resultado
Usuario anónimo	Redirección a inicio de sesión
Responsable accede a su borrador	Permitido
Responsable accede a rendición ajena	Denegado
Responsable aprueba su rendición	Denegado
Revisor observa rendición	Permitido
Usuario de consulta modifica factura	Denegado
Finanzas importa cartola	Permitido
Usuario sin permiso descarga adjunto	Denegado

Caso	Resultado
Administrador reabre proceso	Permitido

10. Criterios de aceptación

- Todos los endpoints requieren autenticación, salvo los expresamente públicos.
- Las acciones críticas requieren permisos personalizados.
- Los botones coinciden con los permisos efectivos.
- Cambiar manualmente una URL no permite evadir la seguridad.
- Un responsable no puede aprobar su propia rendición.
- Los grupos pueden crearse de forma repetible.
- Existen pruebas para casos permitidos y denegados.

A005 — Auditoría e historial

1. Objetivo

Registrar quién realizó una acción, cuándo la realizó, sobre qué registro, qué cambió y por qué.

La Carta Gantt exige que los cambios críticos conserven:

- usuario;
- fecha;
- valor anterior;
- motivo.

2. Cobertura actual

`ModeloAuditoria` cubre:

- fecha de creación;
- fecha de modificación;
- usuario creador;
- usuario modificador.

Esto es suficiente para trazabilidad básica, pero no conserva el detalle de cada cambio.

El modelo `Bitacora` existente registra usuario, acción, modelo, objeto, descripción y fecha.

Sin embargo, le faltan:

- valores anteriores;
- valores nuevos;
- motivo obligatorio;
- dirección IP;
- identificador de solicitud;
- tipo de evento estructurado;

- relación estable con el objeto.

3. Modelo propuesto

```
class EventoAuditoria(models.Model):
    usuario = models.ForeignKey(...)
    accion = models.CharField(...)
    app_label = models.CharField(...)
    modelo = models.CharField(...)
    objeto_pk = models.CharField(...)
    descripcion = models.TextField(...)
    cambios = models.JSONField(default=dict)
    motivo = models.TextField(blank=True)
    fecha = models.DateTimeField(auto_now_add=True)
    ip = models.GenericIPAddressField(null=True, blank=True)
    user_agent = models.TextField(blank=True)
    correlation_id = models.UUIDField(...)
```

Ejemplo del campo `cambios`:

```
{
  "estado": {
    "anterior": "EN_REVISION",
    "nuevo": "APROBADA"
  },
  "monto_aprobado": {
    "anterior": "95000.00",
    "nuevo": "90000.00"
  }
}
```

4. Eventos obligatorios

Se auditarán, como mínimo:

- Presentar rendición.
- Observar rendición.
- Aprobar o rechazar.
- Reabrir.
- Liquidar.
- Registrar pago.
- Anular pago.
- Aplicar movimiento bancario.
- Desactivar aplicación.
- Excluir movimiento.
- Cerrar conciliación.
- Reabrir conciliación.
- Ceder factura a factoring.

- Registrar mora.
- Registrar devolución.
- Descargar documentos sensibles.

5. Estrategia

La auditoría crítica deberá generarse desde los servicios, no exclusivamente desde señales.

Motivo:

- El servicio conoce la acción.
- Conoce el usuario.
- Puede exigir un motivo.
- Conoce el estado anterior.
- Se ejecuta dentro de la misma transacción.

Ejemplo conceptual:

```
Servicio inicia transacción
→ bloquea registro
→ captura estado anterior
→ aplica cambio
→ registra EventoAuditoria
→ confirma transacción
```

6. Vistas

Historial general

```
/administracion/auditoria/
```

Filtros:

- Fecha desde/hasta.
- Usuario.
- Módulo.
- Modelo.
- Acción.
- Objeto.
- Texto libre.

Historial de un objeto

```
/administracion/auditoria/<app>/<modelo>/<pk>/
```

Deberá mostrar una línea de tiempo.

7. Templates

```
templates/administracion/auditoria/  
├─ historial.html  
├─ detalle_evento.html  
└─ timeline_objeto.html
```

8. Permisos

Solo podrán consultar auditoría:

- Administrador.
- Auditor.
- Usuarios con `ver_auditoria`.

Los eventos no se editarán ni eliminarán desde la interfaz normal.

9. Pruebas

- Se crea evento en una transición exitosa.
- No se crea evento si la transacción falla.
- Se registran valores anterior y nuevo.
- El usuario queda registrado.
- El motivo obligatorio es validado.
- Un usuario sin permiso no accede al historial.
- Los eventos no son modificables desde la interfaz.

10. Criterios de aceptación

- Cada operación crítica produce un evento.
- El evento se guarda en la misma transacción.
- Los cambios indican valor anterior y nuevo.
- Los eventos pueden filtrarse.
- El historial de un registro es visible como línea de tiempo.
- Los registros de auditoría son de solo lectura.

A006 — Gestión de archivos adjuntos

1. Objetivo

Crear un mecanismo seguro y reutilizable para subir, validar, almacenar, visualizar y descargar documentos.

La Carta Gantt asigna 8 horas y exige validación, almacenamiento, acceso y descarga segura.

2. Cobertura actual

En Administración existen actualmente:

- Un archivo por importación de cartola.
- Un comprobante por detalle de rendición.

En Extintores existe `ImagenIntervencion` con nueve campos de imagen fijos.

La estructura actual funciona para casos limitados, pero presenta problemas:

- Cantidad fija de archivos.
- Dificultad para reutilizar lógica.
- Falta de metadatos.
- Falta de hash general.
- Falta de control uniforme de descarga.
- Un solo comprobante por gasto.
- No existe historial de descargas.

3. Modelo propuesto

Se utilizará un modelo transversal:

```
class AdjuntoAdministracion(ModeloAuditoria):
    content_type = models.ForeignKey(ContentType, ...)
    object_id = models.PositiveBigIntegerField()
    contenido = GenericForeignKey("content_type", "object_id")

    archivo = models.FileField(upload_to=ruta_adjunto)
    nombre_original = models.CharField(max_length=255)
    tipo_mime = models.CharField(max_length=100)
    extension = models.CharField(max_length=20)
    tamano = models.PositiveBigIntegerField()
    sha256 = models.CharField(max_length=64)
    categoria = models.CharField(max_length=40)
    descripcion = models.CharField(max_length=255, blank=True)
    activo = models.BooleanField(default=True)
```

El uso de relación genérica se limitará mediante una lista blanca de modelos autorizados.

4. Tipos de adjunto iniciales

```
CARTOLA
COMPROBANTE_GASTO
FACTURA
NOTA_CREDITO
RESPALDO_PAGO
CONTRATO_FACTORING
```

LIQUIDACION_FACTORING
DEVOLUCION
OTRO

5. Reglas de validación

Extensiones permitidas

.pdf
.jpg
.jpeg
.png
.xlsx
.xls
.CSV

Las extensiones permitidas dependerán de la categoría.

Validaciones obligatorias

- Tamaño máximo configurable.
- Nombre normalizado.
- Verificación de MIME.
- Extensión autorizada.
- Cálculo SHA-256.
- Rechazo de archivos vacíos.
- Rechazo de archivos ejecutables.
- Prevención de nombres con rutas.
- Control de duplicados cuando corresponda.
- Validación de permiso sobre el objeto padre.

6. Almacenamiento

Los archivos financieros no deberán servirse directamente mediante una URL pública de `MEDIA_URL`.

La descarga deberá pasar por una vista protegida:

`/administracion/adjuntos/<uuid>/descargar/`

Flujo:

Solicitud
→ usuario autenticado
→ validación de permiso
→ búsqueda del adjunto

→ auditoría de descarga
→ FileResponse

7. Servicio

services/adjuntos.py

Funciones previstas:

```
guardar_adjunto(...)  
validar_archivo(...)  
calcular_sha256(...)  
descargar_adjunto(...)  
desactivar_adjunto(...)
```

8. Forms

forms/adjuntos.py

Campos:

- Archivo.
- Categoría.
- Descripción.

El formulario deberá admitir carga múltiple en las pantallas donde sea necesario.

9. Templates

Componente reutilizable:

componentes/adjuntos.html

Debe mostrar:

- Nombre.
- Tipo.
- Tamaño.
- Fecha.
- Usuario que lo cargó.
- Botón de visualizar.
- Botón de descargar.
- Botón de desactivar, según permiso.

10. Migración de archivos existentes

En una primera fase:

- Se conservarán `ImportacionCartola.archivo`.
- Se conservará `DetalleRendicion.comprobante`.
- Los nuevos adjuntos se guardarán en el modelo transversal.
- Posteriormente se ejecutará una migración de datos.
- Los campos antiguos solo se eliminarán después de comprobar la migración.

11. Pruebas

- Archivo válido.
- Extensión inválida.
- MIME incorrecto.
- Archivo demasiado grande.
- Archivo vacío.
- Descarga autorizada.
- Descarga no autorizada.
- Registro de auditoría.
- Carga múltiple.
- Archivo duplicado.
- Desactivación sin borrado físico inmediato.

12. Criterios de aceptación

- Los adjuntos se almacenan fuera del acceso público directo.
- Las extensiones y tamaños se validan.
- Cada adjunto conserva metadatos.
- La descarga exige permiso.
- Las descargas sensibles generan auditoría.
- Se admiten múltiples comprobantes por gasto.
- Los archivos existentes no se pierden.

A007 — Diseño UX y navegación

1. Objetivo

Diseñar una navegación coherente y componentes visuales comunes antes de construir las pantallas finales.

La tarea contempla 12 horas y exige que los flujos y pantallas principales sean aprobados antes de la construcción definitiva.

2. Resultado esperado

Se construirá una plantilla base que defina:

- Encabezado.
- Menú lateral.
- Ruta de navegación.
- Área de contenido.
- Mensajes.
- Botones estándar.
- Tablas.
- Formularios.
- Modales.
- Estados.
- Paginación.
- Diseño responsivo.

3. Navegación principal

```
Administración
├─ Inicio
├─ Rendiciones
│  ├─ Mis rendiciones
│  ├─ Rendiciones por revisar
│  ├─ Liquidaciones
│  └─ Configuración
├─ Banco
│  ├─ Bancos
│  ├─ Cuentas
│  ├─ Importar cartola
│  ├─ Movimientos
│  └─ Conciliaciones
├─ Facturación
│  ├─ Facturas
│  ├─ Pagos
│  ├─ Cobranza
│  └─ Antigüedad de saldos
├─ Factoring
│  ├─ Empresas
│  ├─ Operaciones
│  ├─ Devoluciones
│  └─ Alertas
├─ Reportes
└─ Auditoría
```

El menú deberá construirse según los permisos del usuario.

4. Template base

```
templates/administracion/base_administracion.html
```

Bloques:

```
{% block title %}{% endblock %}
{% block breadcrumbs %}{% endblock %}
{% block page_header %}{% endblock %}
{% block actions %}{% endblock %}
{% block content %}{% endblock %}
{% block extra_js %}{% endblock %}
```

5. Componentes reutilizables

Estado

```
estado_badge.html
```

Colores orientativos:

- Borrador: gris.
- Pendiente: azul.
- En revisión: amarillo.
- Observado: naranja.
- Aprobado o conciliado: verde.
- Rechazado, vencido o crítico: rojo.
- Anulado: gris oscuro.

El color nunca será la única forma de identificar un estado.

Tabla vacía

Debe evitar pantallas completamente en blanco:

```
No existen registros para los filtros seleccionados.
```

Mensajes

- Éxito.
- Advertencia.
- Error.
- Confirmación.
- Información.

Acciones destructivas

Las siguientes acciones requerirán confirmación:

- Anular.
- Reabrir.
- Excluir.
- Rechazar.
- Desactivar adjunto.
- Cerrar conciliación.

6. Pantalla de inicio

URL:

```
/administracion/
```

Vista:

```
AdministracionInicioView
```

Template:

```
administracion/inicio.html
```

Contenido inicial:

- Accesos rápidos.
- Tareas pendientes.
- Rendiciones por revisar.
- Movimientos no conciliados.
- Facturas vencidas.
- Alertas de factoring.

Los indicadores reales se desarrollarán posteriormente. En A007 se implementará el esqueleto visual.

7. Diseño responsivo

La interfaz deberá funcionar en:

- Escritorio.
- Tablet.
- Teléfono.

En pantallas pequeñas:

- El menú lateral será colapsable.

- Las tablas permitirán desplazamiento horizontal.
- Las acciones principales permanecerán visibles.
- Los formularios se mostrarán en una sola columna.
- Los modales no excederán la pantalla.

8. Accesibilidad mínima

- Labels asociados a campos.
- Navegación mediante teclado.
- Indicador de foco.
- Contraste suficiente.
- Mensajes de error junto al campo.
- Iconos acompañados de texto o `aria-label`.
- Tablas con encabezados.
- Confirmaciones comprensibles.

9. JavaScript

El JavaScript se utilizará para:

- Confirmaciones.
- Filtros dinámicos.
- Previsualización de adjuntos.
- Cálculos visuales.
- Carga dependiente de subcategorías.
- Mejora de formularios.

Las reglas críticas no dependerán exclusivamente de JavaScript.

10. Archivos

```
templates/administracion/
├─ base_administracion.html
├─ inicio.html
└─ componentes/
    ├─ breadcrumbs.html
    ├─ mensajes.html
    ├─ estado_badge.html
    ├─ tabla_vacia.html
    ├─ paginacion.html
    └─ modal_confirmacion.html
```

```
static/administracion/
├─ css/administracion.css
└─ js/administracion.js
```

11. Pruebas

- Menú según permisos.
- Renderizado sin autenticación.
- Vista móvil.
- Navegación por teclado.
- Mensajes de formularios.
- Confirmaciones.
- Breadcrumbs.
- Tablas vacías.
- Botones bloqueados por estado.

12. Criterios de aceptación

- Todos los módulos aparecen en una navegación coherente.
 - El menú respeta permisos.
 - Existe una plantilla base reutilizable.
 - Las pantallas funcionan en escritorio y móvil.
 - Estados y mensajes utilizan componentes comunes.
 - La interfaz no permite ejecutar accidentalmente acciones críticas.
 - Los flujos principales han sido revisados antes de construir cada CRUD.
-

4. Resultado consolidado del bloque

Al finalizar A001–A007 deberán existir:

1. Especificación funcional inicial.
2. Glosario de negocio.
3. Matriz de actores.
4. Matriz de estados y transiciones.
5. Modelos revisados.
6. Migraciones verificadas.
7. Matriz de roles y permisos.
8. Auditoría básica y funcional.
9. Gestión segura de adjuntos.
10. Plantilla visual base.
11. Navegación principal.
12. Pruebas transversales iniciales.

5. Definición de terminado del bloque

El bloque completo se considerará terminado cuando:

- El proyecto inicia sin errores.
- Las migraciones están aplicadas.
- Un usuario puede autenticarse.
- Los grupos están configurados.
- Los permisos se verifican en backend.

- Existe una pantalla inicial de Administración.
- Los menús cambian según el usuario.
- Los cambios críticos pueden generar auditoría.
- Un archivo puede cargarse y descargarse de forma protegida.
- La estructura está preparada para comenzar Banco, Facturación y Rendiciones.
- Los tests transversales pasan.